

DAY 14 — Advanced SQL Practice + Debugging + Assessment

Today is assessment day. No major new theory.

You will combine everything from Days 1–13:

- SELECT
- WHERE
- ORDER BY
- LIMIT
- DISTINCT
- Aggregations
- GROUP BY
- HAVING
- CASE
- INNER JOIN
- LEFT JOIN
- Multiple JOINS
- Subqueries
- CTEs
- Window functions
- ROW_NUMBER
- RANK
- DENSE_RANK
- LAG
- Query debugging
- RAG/data-engineering analytics

Important: Don't look for answers online or ask me for solutions before attempting. Send me your SQL attempts and I will review/debug them.

1. Assessment Database

Use this schema mentally for all questions.

users

id	name	country	account_status
1	Aarav	India	active
2	Emma	USA	active

3	Liam	UK	inactive
4	Sofia	India	active
5	Noah	USA	suspended
6	Olivia	Canada	active

documents

id	user_id	title	file_type	page_count	status
101	1	RAG Fundamentals	pdf	20	processed
102	1	PostgreSQL Guide	pdf	45	processed
103	2	Python Basics	pdf	30	processing
104	3	FastAPI Tutorial	md	15	failed
105	4	Vector Databases	pdf	50	processed
106	5	Docker Guide	pdf	25	processing
107	6	LLM Fundamentals	pdf	60	processed
108	6	Prompt Engineering	md	18	processed
109	2	Embeddings Guide	pdf	55	failed
110	4	RAG Evaluation	pdf	35	processing

document_chunks

Assume:

id

document_id

chunk_number

content

embeddings

Assume:

id

chunk_id

model

status

Possible embedding statuses:

success

failed

processing

Relationships:

users.id

↓

documents.user_id

↓

document_chunks.document_id

↓

embeddings.chunk_id

PART A — SQL Fundamentals Review

Q1 — Filtering

Return all active users from India.

Q2 — Sorting

Return all documents ordered by:

1. **page_count descending**
 2. **id ascending as the tie-breaker**
-

Q3 — Top N

Return the 3 largest documents by page count.

Output:

id

title

page_count

Q4 — DISTINCT

Return all unique document file types.

PART B — Aggregation

Q5 — Basic aggregation

Find:

total_documents

total_pages

average_pages

largest_document

smallest_document

from documents.

Q6 — GROUP BY

Find the number of documents for each status.

Expected conceptual output:

status | document_count

Q7 — GROUP BY + HAVING

Find users who have uploaded at least 2 documents.

Output:

user_id

document_count

Q8 — CASE

Create a size category for every document:

page_count < 20 → Small

20–50 → Medium

> 50 → Large

Output:

id

title

page_count

size_category

Be careful about your CASE condition ordering.

PART C — JOIN Assessment

Q9 — INNER JOIN

Return:

user_name

document_title

page_count

for every document that belongs to a user.

Q10 — LEFT JOIN

Return every user, including users who have uploaded zero documents.

Output:

user_id

name

document_count

Important

Think carefully about:

COUNT(*)

versus:

COUNT(d.id)

Q11 — Users with no documents

Find users who have no documents.

Hint:

You should think about:

LEFT JOIN

+

IS NULL

PART D — Multiple JOINS

Q12 — RAG pipeline counts

Using:

users

documents

document_chunks

produce:

user_id

user_name

document_count

chunk_count

Return every user, even if they have no documents/chunks.

Engineering warning

Joining multiple one-to-many relationships can multiply rows.

Don't blindly do:

```
COUNT(d.id)
```

and assume it is correct.

Think about:

```
COUNT(DISTINCT ...)
```

or pre-aggregation.

PART E — Subqueries

Q13 — Above-average documents

Find documents whose `page_count` is greater than the average page count across all documents.

Output:

`id`

`title`

`page_count`

You should use a subquery.

Q14 — Largest document

Find the document(s) having the maximum `page_count`.

Do not simply use:

```
ORDER BY page_count DESC
```

```
LIMIT 1
```

For this question, think about what happens if two documents tie for maximum size.

Q15 — Users above average document count

First determine each user's document count.

Then return users whose document count is greater than the average document count per user.

This is a good place to use a subquery or CTE.

PART F — CTE Assessment

Q16 — Document statistics CTE

Create a CTE that calculates:

`user_id`

`document_count`

`total_pages`

average_pages

Then return users having:

total_pages > 50

Output:

user_id

document_count

total_pages

average_pages

Q17 — Multiple CTEs

Build two CTEs:

CTE 1

Calculate document statistics:

user_id

document_count

total_pages

CTE 2

Calculate document status counts:

user_id

processed_documents

failed_documents

processing_documents

Then combine them.

Final output:

user_id

document_count

total_pages

processed_documents

failed_documents

processing_documents

PART G — Window Functions

Q18 — ROW_NUMBER

Assign a unique row number to documents ordered by:

page_count DESC

Output:

id

title

page_count

row_number

Q19 — Rank documents per user

Rank each user's documents by page count.

Use:

RANK()

and:

PARTITION BY user_id

Output:

user_id

document_id

title

page_count

rank

Q20 — Top 2 documents per user

Return the 2 largest documents for every user.

You must use:

ROW_NUMBER()

+

PARTITION BY

+

CTE

Expected columns:

user_id

document_id

title

page_count

rn

Q21 — RANK vs DENSE_RANK

Suppose page counts are:

100

100

90

80

80

70

Write down the result of:

RANK()

and

DENSE_RANK()

Don't run SQL.

Q22 — Running total

Order documents by id and calculate a running total of page_count.

Output:

id

title

page_count

running_pages

Use:

SUM(...) OVER (...)

Q23 — LAG

For each document ordered by id, show:

id

title

page_count

previous_page_count

difference_from_previous

Use LAG().

PART H — ADVANCED RAG ENGINEERING CHALLENGE

This is the most important part of today's assessment.

Q24 — RAG Pipeline Health Dashboard

Build a query producing approximately:

document_id

document_title

user_name

page_count

chunk_count

embedding_count

successful_embeddings

failed_embeddings

embedding_success_rate

You should use:

- users
- documents
- document_chunks
- embeddings
- CTEs
- LEFT JOIN
- GROUP BY
- conditional aggregation
- CASE

Important engineering requirement

A document may have:

many chunks

and each chunk may have:

many embedding records

Therefore, joining everything directly can cause:

row multiplication

A production-minded solution should consider pre-aggregating child tables in separate CTEs.

PART I — Ranking Challenge

Q25 — Rank documents by pipeline quality

Using the result from Q24, rank documents based on their embedding success rate.

Output:

document_id

document_title

embedding_success_rate

pipeline_rank

Use:

RANK()

or DENSE_RANK().

Explain why you chose one.

PART J — DEBUGGING

These are intentionally broken.

Your job is to identify:

1. What's wrong?

2. Why is it wrong?

3. How would you fix it?

Don't immediately rewrite the query. Explain the bug first.

Debugging Q26 — LEFT JOIN bug

```
SELECT
u.id,
u.name,
COUNT(d.id) AS document_count
FROM users u
LEFT JOIN documents d
ON u.id = d.user_id
WHERE d.status = 'processed'
GROUP BY u.id, u.name;
```

Question

Does this still behave like a true LEFT JOIN for users with no documents?

Explain why or why not.

Debugging Q27 — Window function bug

```
SELECT
d.id,
d.user_id,
d.title,
ROW_NUMBER() OVER (
```

ORDER BY d.page_count DESC

) AS rn

FROM documents d

WHERE rn <= 2;

Identify both problems.

Debugging Q28 — CASE bug

SELECT

id,

page_count,

CASE

WHEN page_count > 20 THEN 'Medium'

WHEN page_count > 50 THEN 'Large'

ELSE 'Small'

END AS size_category

FROM documents;

What happens to a document with:

page_count = 60

Why?

Debugging Q29 — Multiple JOIN bug

Suppose:

Document A

↓

10 chunks

↓

2 embeddings per chunk

The query:

SELECT

d.id,

COUNT(c.id) AS chunk_count,

COUNT(e.id) AS embedding_count

FROM documents d

LEFT JOIN document_chunks c

ON d.id = c.document_id

LEFT JOIN embeddings e

ON c.id = e.chunk_id

GROUP BY d.id;

produces incorrect chunk_count.

Explain:

Why does this happen?

And give one strategy for fixing it.

Debugging Q30 — Scalar subquery bug

```
SELECT
id,
title
FROM documents
WHERE page_count = (
SELECT page_count
FROM documents
);
```

What is wrong?

What type of subquery would you need instead?

PART K — MINI PROJECT

Q31 — Build a RAG User Analytics Dashboard

Create one SQL query that returns one row per user:

user_id
user_name
document_count
total_pages
average_pages
processed_documents
failed_documents
processing_documents

Then add:

pipeline_status

with rules:

No documents

→ 'NO_DOCUMENTS'

Any failed document

→ 'ATTENTION'

All documents processed

→ 'HEALTHY'

Otherwise

→ 'IN_PROGRESS'

Then add:

user_rank

ranking users by:

total_pages DESC

Requirements

Your query should demonstrate:

- CTE
- LEFT JOIN
- GROUP BY
- COUNT
- SUM
- AVG
- conditional aggregation
- CASE
- window function
- RANK() or DENSE_RANK()

PART L — INTERVIEW ASSESSMENT

Answer these without searching.

Q32

What's the difference between:

GROUP BY

and:

PARTITION BY

Q33

Why can't you normally do this?

SELECT

ROW_NUMBER() OVER (...) AS rn

FROM documents

WHERE rn <= 3;

Q34

Explain the difference between:

INNER JOIN

LEFT JOIN

FULL JOIN

in practical Data Engineering terms.

Q35

Why can multiple JOINS cause incorrect aggregate counts?

Q36

When would you choose a CTE instead of a deeply nested subquery?

Q37

What is the difference between:

ROW_NUMBER

RANK

DENSE_RANK

Q38

You're processing 500 million staging records and need the latest record for each document_id.

Which SQL technique would you use?

Explain your reasoning.

PART M — EXPLANATION CHALLENGE

Explain this query line by line:

```
WITH ranked_documents AS (  
  SELECT  
    d.id AS document_id,  
    d.user_id,  
    d.title,  
    d.page_count,  
    ROW_NUMBER() OVER (  
      PARTITION BY d.user_id  
      ORDER BY d.page_count DESC, d.id  
    ) AS rn  
  FROM documents d  
)  
SELECT  
  document_id,
```

```

user_id,
title,
page_count,
rn
FROM ranked_documents
WHERE rn <= 2
ORDER BY user_id, rn;

```

Your explanation must cover:

- 1. What the CTE does**
- 2. Why PARTITION BY exists**
- 3. What the window ORDER BY does**
- 4. Why d.id is included**
- 5. What ROW_NUMBER() does**
- 6. Why WHERE rn <= 2 is outside the CTE**
- 7. What "top 2 per user" means**
- 8. What would change if we used RANK() instead**

FINAL TEST — 100 POINTS

Section Points

Fundamentals 10

Aggregation 10

JOINS 15

Subqueries 10

CTEs 10

Window functions 15

Debugging 15

RAG engineering 10

Interview concepts 5

Total 100

Target

90–100 → Excellent

80–89 → Strong

70–79 → Acceptable

60–69 → Needs improvement

<60 → Repeat advanced SQL practice

But don't focus only on the score. A single serious misunderstanding—especially around joins, aggregation, NULLs, or window functions—matters more than getting a high score through memorization.

Your submission format

Don't solve all 38 questions at once if that's too much.

Start with:

Q1:

<your SQL>

Q2:

<your SQL>

...

Q10:

<your SQL>

Then send them to me.

I will review your SQL like a Data Engineering code review:

- Correct
- Works but can be improved
- Incorrect
- Bug explanation
- Concept you're missing
- Production consideration

Since you haven't submitted your previous Day 1–13 solutions, I cannot yet identify recurring SQL mistakes or reliably assess mastery. Day 14 is the point where we'll start measuring that from your actual attempts.